

Федеральное государственное автономное образовательное учреждение  
высшего образования «Национальный исследовательский университет  
ИТМО»

Проект: «*Investment++*»

Выполнили студенты:

Кузьмин Артемий Андреевич

Молодиченко Семен Андреевич

Силинцев Владислав Витальевич

Шаматульский Роман Константинович

Самсонов Демьян Александрович

Антонов Никита Сергеевич

Ребрый Егор Сергеевич

Куручка Дмитрий Сергеевич

Руководитель:

Ключев Аркадий Олегович

г. Санкт-Петербург, 2026 г.

# Оглавление

|                                                                  |           |
|------------------------------------------------------------------|-----------|
| <b>Оглавление.....</b>                                           | <b>2</b>  |
| <b>Введение.....</b>                                             | <b>4</b>  |
| Актуальность работы.....                                         | 4         |
| Цель работы.....                                                 | 4         |
| Задачи работы.....                                               | 5         |
| <b>Техническое задание.....</b>                                  | <b>6</b>  |
| 1 Введение.....                                                  | 6         |
| 2 Основания для разработки.....                                  | 6         |
| 3 Назначение разработки.....                                     | 6         |
| 3.1 Функциональное назначение.....                               | 6         |
| 3.2 Эксплуатационное назначение.....                             | 6         |
| 4 Требования к программе или программному изделию.....           | 7         |
| 4.1 Требования к функциональным характеристикам.....             | 7         |
| 4.2 Требования к надежности.....                                 | 7         |
| 4.3 Условия эксплуатации.....                                    | 7         |
| 4.4 Требования к составу и параметрам технических средств.....   | 8         |
| 4.5 Требования к информационной и программной совместимости..... | 8         |
| 4.6 Требования к маркировке и упаковке.....                      | 8         |
| 4.7 Требования к транспортированию и хранению.....               | 8         |
| 4.8 Специальные требования.....                                  | 8         |
| 5 Требования к программной документации.....                     | 9         |
| 6 Техничко-экономические показатели.....                         | 9         |
| 7 Стадии и этапы разработки.....                                 | 9         |
| 8 Порядок контроля и приемки.....                                | 9         |
| 9 Приложения.....                                                | 10        |
| <b>Архитектура системы.....</b>                                  | <b>11</b> |
| 1. Мобильное Android-приложение.....                             | 11        |
| 2. Кроссплатформенное мобильное приложение.....                  | 11        |
| 3. Сервис взаимодействия с клиентами.....                        | 12        |
| 4. Сервис работы с базой данных.....                             | 12        |
| 5. Сервис имитации клиентской нагрузки.....                      | 12        |
| 6. Сервис получения котировок.....                               | 13        |
| 7. Linux-драйвер имитации котировок.....                         | 13        |
| 8. Контейнеризация и развертывание.....                          | 13        |
| 9. Документация проекта.....                                     | 14        |
| Взаимодействие компонентов.....                                  | 14        |
| <b>Описание реализации.....</b>                                  | <b>15</b> |
| 1. Мобильное приложение (Android / Kotlin).....                  | 15        |

|                                                           |           |
|-----------------------------------------------------------|-----------|
| 2. Backend сервис (3-connection-app).....                 | 15        |
| 3. Сервис работы с базой данных (4-DB-connection).....    | 16        |
| 4. Redis (кэш и обмен данными).....                       | 17        |
| 5. Сервис котировок (6-quotes-receiver).....              | 17        |
| 6. Linux-драйвер имитации котировок (7-linux-driver)..... | 17        |
| 7. Сервис имитации нагрузки (5-connection-imitator).....  | 18        |
| 8. Общие принципы реализации.....                         | 18        |
| <b>Тестирование.....</b>                                  | <b>19</b> |
| Модульное.....                                            | 19        |
| Интеграционное.....                                       | 19        |
| Системное.....                                            | 20        |
| <b>Заключение.....</b>                                    | <b>21</b> |
| <b>Список литературы.....</b>                             | <b>22</b> |

## **Введение**

### **Актуальность работы**

В последние годы интерес к инвестициям и трейдингу значительно вырос. Всё больше людей используют мобильные приложения для управления своими финансами, покупки акций, анализа рынка и отслеживания состояния инвестиционного портфеля. Такие приложения позволяют быстро получать доступ к финансовой информации и совершать операции в удобном формате.

Разработка подобных систем является актуальной задачей, поскольку современные финансовые сервисы представляют собой не просто мобильное приложение, а набор взаимосвязанных компонентов: клиентскую часть, серверную логику, базы данных и отдельные сервисы для обработки различных операций. Создание учебного проекта в данной предметной области позволяет получить практический опыт разработки приложений, приближенных к реальным коммерческим продуктам.

В рамках данного проекта разрабатывается программное обеспечение для трейдинга и инвестиций, моделирующее базовую работу инвестиционной платформы.

### **Цель работы**

Целью работы является разработка минимально жизнеспособного продукта (MVP) — мобильного приложения для трейдинга и инвестиций с backend-частью и набором сервисов, реализующих основную бизнес-логику системы.

Разрабатываемое приложение должно демонстрировать основные возможности подобных платформ: работу с пользовательскими данными, отображение информации об активах, выполнение операций и взаимодействие с серверной частью.

## Задачи работы

Для достижения поставленной цели необходимо решить следующие задачи:

- изучить особенности существующих приложений для трейдинга и инвестиций;
- определить основной функционал разрабатываемой системы;
- спроектировать архитектуру приложения и backend-части;
- разработать мобильное приложение для платформы Android;
- реализовать серверную часть для обработки запросов и хранения данных;
- разработать отдельные сервисы, моделирующие работу ключевых модулей системы;
- реализовать взаимодействие между клиентом и сервером;
- подготовить документацию по проекту.

# **Техническое задание**

## **1 Введение**

Настоящее техническое задание распространяется на разработку программного изделия «Система мобильного инвестирования». Документ устанавливает назначение, требования к функциональным и эксплуатационным характеристикам, составу документации, стадиям выполнения работ и порядку приемки.

Разработка выполняется для учебно-практических целей с возможностью последующего использования решения как демонстрационного прототипа инвестиционной платформы.

## **2 Основания для разработки**

Основанием для разработки являются: учебный план и задание по дисциплине «Разработка мобильных приложений»; методические требования к проекту микросервисной системы; решение о выполнении темы «Система мобильного инвестирования».

Наименование организации-разработчика и реквизиты договора в рамках учебного проекта не устанавливаются.

## **3 Назначение разработки**

### **3.1 Функциональное назначение**

Программное изделие предназначено для получения и отображения биржевых котировок, ведения пользовательского инвестиционного портфеля, выполнения операций покупки и продажи финансовых инструментов, а также предоставления сводной аналитической информации по активам пользователя.

### **3.2 Эксплуатационное назначение**

Программное изделие предназначено для эксплуатации в составе клиент-серверной системы: мобильный клиент на устройствах Android/iOS и

серверная часть в контейнерной инфраструктуре. Система используется в учебных, исследовательских и демонстрационных целях.

## **4 Требования к программе или программному изделию**

### **4.1 Требования к функциональным характеристикам**

Система должна обеспечивать: регистрацию и аутентификацию пользователя; получение рыночных данных от внешних источников; отображение котировок и истории изменения цен; формирование и отображение структуры портфеля; выполнение операций покупки/продажи в пределах доступного баланса; хранение истории операций; выдачу уведомлений о статусе операций.

Целевая производительность: одновременное обслуживание до 10 000 клиентских сессий при сохранении работоспособности основных пользовательских сценариев.

### **4.2 Требования к надежности**

Система должна обеспечивать отказоустойчивую работу серверных сервисов, обработку ошибок с записью в журналы, автоматический перезапуск контейнеров при сбоях и сохранность пользовательских данных при штатных и нештатных завершениях работы.

Допустимое время недоступности при отказе отдельного сервиса не должно превышать 5 минут при наличии штатного механизма оркестрации.

### **4.3 Условия эксплуатации**

Серверная часть должна эксплуатироваться под управлением Linux в среде контейнеризации Docker/Kubernetes. Клиентская часть должна эксплуатироваться на мобильных устройствах Android версии 10 и выше и iOS версии 15 и выше.

Эксплуатация предусматривает круглосуточный режим работы серверной части с проведением регламентных работ по утвержденному графику.

#### **4.4 Требования к составу и параметрам технических средств**

Минимальный состав серверных средств: не менее 4 vCPU, 8 ГБ ОЗУ и 100 ГБ дискового пространства для стенда разработки/демонстрации. Для масштабирования производственной среды параметры увеличиваются исходя из целевой нагрузки.

Клиентские устройства должны иметь стабильное подключение к сети Интернет по Wi-Fi или мобильной сети передачи данных.

#### **4.5 Требования к информационной и программной совместимости**

Взаимодействие компонентов должно выполняться по протоколу HTTPS с использованием REST API и формата JSON. Система должна поддерживать работу с реляционной СУБД PostgreSQL и кэшированием в Redis.

Серверные компоненты реализуются на языках Go и C (в части расчетных модулей), клиентские компоненты — на Kotlin (Android) и Swift/SwiftUI (iOS) либо кроссплатформенном фреймворке по согласованию.

#### **4.6 Требования к маркировке и упаковке**

Для программного изделия требования к маркировке и упаковке не предъявляются. Поставка выполняется в виде электронного дистрибутива и репозитория исходных текстов.

#### **4.7 Требования к транспортированию и хранению**

Транспортирование программного изделия осуществляется по электронным каналам связи. Хранение дистрибутива и исходных кодов должно обеспечиваться в системе контроля версий и в архивных копиях на защищенном носителе.

#### **4.8 Специальные требования**

Должны быть обеспечены: шифрование трафика (TLS), аутентификация и авторизация на основе JWT, разграничение прав доступа, защита от базовых

веб-уязвимостей, резервное копирование критичных данных и возможность аудита действий пользователей.

## **5 Требования к программной документации**

В состав программной документации должны входить: техническое задание; описание программы; описание применения; руководство пользователя; руководство администратора; программа и методика испытаний; описание интерфейса программирования приложений (API).

Документация должна поставляться в электронном виде в форматах DOCX/PDF и храниться совместно с версиями программного изделия.

## **6 Технико-экономические показатели**

Ожидаемые технико-экономические показатели: сокращение времени получения пользователем рыночной информации; снижение затрат на разработку за счет использования открытых технологий; уменьшение стоимости масштабирования за счет микросервисной архитектуры и контейнеризации; повторное использование решения как учебно-исследовательской платформы.

## **7 Стадии и этапы разработки**

Разработка выполняется по стадиям: анализ предметной области и требований; проектирование архитектуры; разработка серверной части; разработка мобильного клиента; интеграция компонентов; тестирование; подготовка документации; ввод в эксплуатацию (демонстрационная приемка).

Состав этапов и календарные сроки уточняются календарным планом проекта.

## **8 Порядок контроля и приемки**

Контроль выполнения требований технического задания проводится посредством функциональных, интеграционных, нагрузочных и приемочных испытаний по утвержденной программе и методике испытаний.

Критериями приемки являются: соответствие реализованных функций требованиям раздела 4; отсутствие критических дефектов по результатам испытаний; наличие полного комплекта программной документации по разделу 5.

Приемка результатов разработки оформляется протоколом испытаний и актом приемки в рамках учебного проекта.

## **9 Приложения**

В приложения включаются: схема архитектуры системы; спецификация API; перечень используемых технологий и внешних сервисов; диаграммы взаимодействия компонентов; примеры пользовательских сценариев.

## Архитектура системы

Проект представляет собой экосистему сервисов для моделирования работы инвестиционной платформы. Архитектура построена по модульному принципу и включает клиентские приложения, backend-сервисы, сервисы получения данных и вспомогательные компоненты.

Структура проекта разделена на несколько независимых модулей, каждый из которых отвечает за отдельную часть системы.

### 1. Мобильное Android-приложение

Модуль: 1-android-app.

Данный модуль содержит нативное мобильное приложение для платформы Android. Его основная задача — предоставить пользователю интерфейс для взаимодействия с инвестиционной системой.

Основные функции:

- авторизация пользователя;
- просмотр списка активов и котировок;
- отображение инвестиционного портфеля;
- выполнение операций покупки и продажи активов;
- просмотр информации о балансе и истории операций.

### 2. Кроссплатформенное мобильное приложение

Модуль: 2-cross-platform-app.

Содержит кроссплатформенную версию мобильного приложения, реализованную с использованием React Native.

Назначение:

- обеспечение работы приложения на нескольких мобильных платформах;
- реализация аналогичного функционала нативному приложению.

### **3. Сервис взаимодействия с клиентами**

Модуль: 3-connection-app.

Представляет собой backend-сервис, обеспечивающий взаимодействие мобильных приложений с серверной инфраструктурой.

Функции:

- прием и обработка HTTP/API-запросов;
- маршрутизация запросов к внутренним сервисам;
- обработка пользовательских операций.

### **4. Сервис работы с базой данных**

Модуль: 4-DB-connection.

Отвечает за взаимодействие backend-сервисов с системой хранения данных.

Функции:

- сохранение пользовательских данных;
- хранение информации о портфелях и операциях;
- получение и обновление данных по запросам других сервисов.

### **5. Сервис имитации клиентской нагрузки**

Модуль: 5-connection-imitator.

Используется для тестирования системы под нагрузкой и моделирования поведения большого количества клиентов.

Функции:

- генерация случайных API-запросов;
- имитация активности пользователей;
- тестирование устойчивости системы.

## **6. Сервис получения котировок**

Модуль: 6-quotes-receiver.

Отвечает за получение или генерацию данных о рыночных котировках.

Функции:

- получение данных о стоимости активов;
- передача котировок backend-сервисам;
- обновление информации для клиентских приложений.

## **7. Linux-драйвер имитации котировок**

Модуль: 7-linux-driver.

Низкоуровневый модуль, реализующий генерацию или эмуляцию данных котировок на уровне Linux.

Назначение:

- моделирование внешнего источника рыночных данных;
- интеграция с сервисом получения котировок.

## **8. Контейнеризация и развертывание**

Модуль: docker.

Содержит конфигурацию контейнеров и сценарии запуска компонентов системы.

Назначение:

- упрощение развертывания проекта;
- стандартизация окружения разработки.

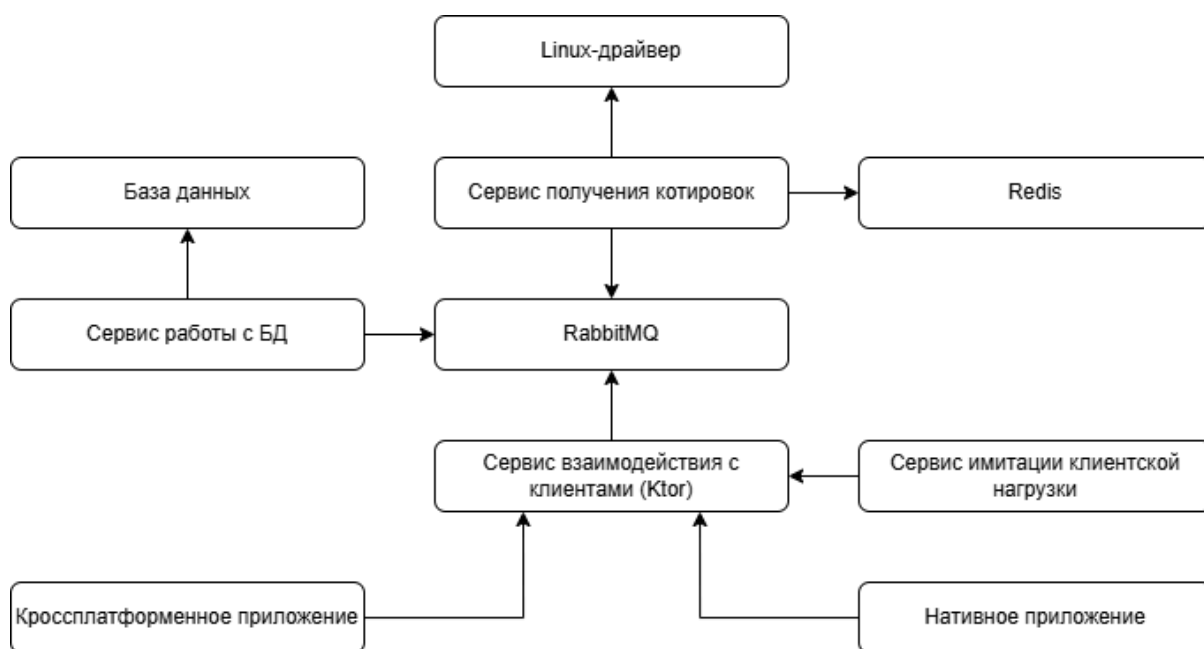
## 9. Документация проекта

Модуль: docs.

Содержит документацию по проекту, описания архитектуры, инструкции по запуску и технические материалы.

### Взаимодействие компонентов

Работа системы организована следующим образом:



Для тестирования системы используется сервис имитации клиентской нагрузки, генерирующий запросы к backend-инфраструктуре.

## Описание реализации

Реализация проекта построена на разделении системы на несколько независимых сервисов, каждый из которых отвечает за свою часть логики. Основной принцип — разделение ответственности и взаимодействие через HTTP API и in-memory хранилище Redis.

### 1. Мобильное приложение (Android / Kotlin)

Мобильное приложение реализовано на Kotlin с использованием Jetpack Compose. Оно является клиентской частью системы и отвечает только за отображение данных и отправку запросов на сервер.

Основные компоненты:

- UI слой (Compose) — экраны приложения (портфель, список активов, детали акции, профиль пользователя);
- ViewModel слой — хранит состояние экранов и управляет логикой UI;
- Repository слой — отвечает за получение данных с backend через HTTP;
- Network layer — реализация API-запросов (Retrofit или аналог).

Логика работы:

1. Пользователь открывает экран.
2. ViewModel запрашивает данные через Repository.
3. Repository отправляет запрос в backend (3-connection-app).
4. Полученные данные отображаются в UI.

### 2. Backend сервис (3-connection-app)

Этот модуль является основным API-шлюзом системы.

Функции:

- обработка запросов от мобильных приложений;
- маршрутизация запросов к другим сервисам (БД, Redis, quotes);
- агрегация данных для клиента.

Реализация:

Backend реализован как REST API (Kotlin + Ktor), где каждый endpoint соответствует конкретной бизнес-операции:

- /login
- /portfolio
- /stocks
- /trade/buy
- /trade/sell

Backend не хранит бизнес-данные напрямую, а получает их из:

- 4-DB-connection (долгосрочные данные)
- Redis (быстрые данные)
- 6-quotes-receiver (котировки)

### **3. Сервис работы с базой данных (4-DB-connection)**

Этот сервис отвечает за взаимодействие с PostgreSQL.

Хранимые данные:

- пользователи;
- портфели;
- транзакции;
- история операций.

Реализация:

- SQL-запросы выполняются напрямую (без тяжелых ORM);
- сервис предоставляет API для backend:
  - getUser
  - updateBalance
  - getPortfolio
  - saveTransaction

## 4. Redis (кэш и обмен данными)

Redis используется как высокоскоростное хранилище временных данных и механизм обмена сообщениями между сервисами.

Использование Redis:

1. Кэширование котировок
  - a. последние цены акций хранятся в Redis;
  - b. backend читает данные напрямую без обращения к БД.
2. Кэш пользовательских данных
  - a. баланс;
  - b. портфель;
  - c. часто запрашиваемые данные.
3. Pub/Sub (обновления котировок)
  - a. сервис котировок публикует новые цены;
  - b. backend подписан и получает обновления в реальном времени.

## 5. Сервис котировок (6-quotes-receiver)

Отвечает за получение и обновление данных о стоимости акций.

Реализация:

- получает данные от 7-linux-driver;
- нормализует и обрабатывает котировки;
- записывает актуальные значения в Redis;
- при изменениях публикует события через Redis Pub/Sub.

## 6. Linux-драйвер имитации котировок (7-linux-driver)

Низкоуровневый модуль, который генерирует поток данных о котировках.

Реализация:

- написан на C;
- имитирует изменения цен в реальном времени;

- отправляет данные в сервис котировок через системный интерфейс (сокеты/pipe).

## **7. Сервис имитации нагрузки (5-connection-imitator)**

Используется для тестирования системы.

Реализация:

- генерирует случайные API-запросы;
- имитирует действия пользователей:
- просмотр акций;
- покупка/продажа;
- обновление портфеля;
- позволяет проверить стабильность backend под нагрузкой.

## **8. Общие принципы реализации**

В проекте используются следующие подходы:

- разделение системы на независимые сервисы;
- взаимодействие через REST API;
- использование Redis для ускорения доступа к данным;
- хранение критичных данных в PostgreSQL;
- генерация котировок как отдельного внешнего источника;
- имитация нагрузки для проверки устойчивости системы.

## Тестирование

В рамках проекта использовались три основных уровня тестирования: модульное, интеграционное и системное. Такой подход позволяет проверить как отдельные компоненты системы, так и их взаимодействие в рамках всей архитектуры.

### Модульное

Модульное тестирование применялось для проверки отдельных классов и компонентов внутри сервисов.

Основное внимание уделялось:

- проверке корректности бизнес-логики (расчёт портфеля, обработка операций покупки/продажи);
- валидации методов работы с данными;
- проверке утилитарных функций (форматирование данных, преобразование моделей).

Тесты реализованы с использованием стандартных инструментов IntelliJ IDEA и встроенных фреймворков тестирования для Kotlin (JUnit).

Цель данного уровня тестирования — убедиться, что отдельные компоненты системы корректно работают изолированно от остальной инфраструктуры.

### Интеграционное

Интеграционное тестирование использовалось для проверки взаимодействия между отдельными сервисами системы.

Проверялись следующие сценарии:

- взаимодействие мобильного приложения с backend API (3-connection-app);

- корректность работы backend при обращении к сервису базы данных (4-DB-connection);
- получение и кэширование котировок через Redis;
- передача данных от сервиса котировок (6-quotes-receiver) к backend-системе.

Для интеграционного тестирования было разработано отдельное тестовое инструментальное обеспечение на языке Kotlin (частично Python), которое имитирует работу клиентских приложений и отправляет HTTP-запросы к сервисам системы.

Инструментальные тесты позволяют проверить корректность взаимодействия компонентов без участия пользовательского интерфейса.

## **Системное**

Системное тестирование проводилось для проверки работы всей системы в целом, включая все сервисы и их взаимодействие.

В рамках данного этапа моделировались реальные сценарии использования приложения:

- авторизация пользователя;
- получение списка акций и котировок;
- просмотр портфеля;
- выполнение операций покупки и продажи активов;
- обновление данных в реальном времени.

Для системного тестирования использовался сервис имитации нагрузки (5-connection-imitator), который генерирует большое количество случайных запросов к backend-системе, моделируя поведение реальных пользователей.

Дополнительно проверялась устойчивость системы при увеличении количества одновременных запросов и корректность работы Redis-кэша и базы данных под нагрузкой.

## **Заключение**

В ходе выполнения проекта было разработано минимально жизнеспособное программное обеспечение для трейдинга и инвестиций, включающее мобильное приложение и backend-часть.

В процессе работы были изучены особенности существующих инвестиционных платформ и определены основные функции, необходимые для реализации базового инвестиционного сервиса. На основе проведенного анализа была спроектирована архитектура системы с разделением на клиентскую и серверную части.

Было разработано мобильное приложение для платформы Android, реализующее базовые пользовательские сценарии взаимодействия с системой. Также была реализована backend-часть, обеспечивающая обработку пользовательских запросов, хранение данных и взаимодействие с клиентским приложением.

Дополнительно были разработаны отдельные сервисы, моделирующие работу ключевых модулей инвестиционной платформы, что позволило приблизить структуру проекта к архитектуре реальных приложений данного класса.

В результате выполнения работы был получен практический опыт проектирования и разработки клиент-серверных приложений, организации взаимодействия между несколькими сервисами, а также подготовки технической документации в соответствии с установленными требованиями.

## Список литературы

1. ГОСТ 7.32–2017. Система стандартов по информации, библиотечному и издательскому делу. Отчет о научно-исследовательской работе. Структура и правила оформления. — Текст : электронный // Электронный фонд правовых и нормативно-технических документов [сайт]. — URL: <https://docs.cntd.ru/document/1200157208> (дата обращения: 14.05.2026).
2. ГОСТ 19.201–78. Единая система программной документации. Техническое задание. Требования к содержанию и оформлению. — Текст : электронный // Электронный фонд правовых и нормативно-технических документов [сайт]. — URL: <https://docs.cntd.ru/document/1200007648> (дата обращения: 14.05.2026).
3. ГОСТ Р 7.0.80–2023. Система стандартов по информации, библиотечному и издательскому делу. Библиографическая запись. Заголовок. Общие требования и правила составления. — Текст : электронный // Электронный фонд правовых и нормативно-технических документов [сайт]. — URL: <https://docs.cntd.ru/document/1301694709> (дата обращения: 14.05.2026).